

Resumo detalhado - Aquisição e integração universal de dados com YOLOv8 e OCR

Baseado no artigo: Multi-device Universal Automation Data Acquisition and Integration System

Visão geral: o trabalho propõe um sistema universal de aquisição e integração de dados industriais que usa YOLOv8 para localizar regiões relevantes em telas/imagens e OCR para converter o conteúdo visual em texto ou números. O objetivo é reduzir a dependência de drivers, APIs e protocolos específicos de cada equipamento.

1. Tema e objetivo do trabalho

O artigo Multi-device Universal Automation Data Acquisition and Integration System apresenta um sistema para aquisição e integração automática de dados provenientes de equipamentos industriais heterogêneos.

O problema central é a dificuldade de integrar dispositivos de diferentes fabricantes, gerações, interfaces e protocolos. Em ambientes industriais, cada equipamento pode exigir um driver, uma API, um protocolo específico ou até desenvolvimento customizado.

A solução proposta desloca a integração do nível de protocolo para o nível visual: se o dado aparece na tela de um equipamento ou software, ele pode ser capturado como imagem e processado automaticamente.

- YOLOv8: localiza a região da imagem onde está o dado desejado.
- OCR: converte o conteúdo da região localizada em caracteres, números e unidades.
- Servidor: concentra a carga computacional e permite que dispositivos antigos ou pouco potentes participem do sistema.

2. Contexto: aquisição de dados na Indústria 4.0

O artigo parte da ideia de que dados são fundamentais para análise, controle, automação, manutenção e tomada de decisão. Por isso, a aquisição de dados (DAQ) é tratada como etapa inicial de praticamente qualquer aplicação orientada a dados.

Tradicionalmente são adquiridas grandezas como tensão, corrente, temperatura, pressão e som. O fluxo clássico é: fenômeno físico -> sensor/atuador -> hardware de aquisição -> software -> análise -> aplicação.

3. Aquisição baseada em hardware

A aquisição por hardware funciona bem quando o sinal físico pode ser acessado diretamente por sensores e condicionamento apropriado.

Entretanto, quando o dado desejado não está disponível em uma interface padrão, passa a ser necessário desenvolver hardware, firmware ou circuitos específicos, aumentando custo, tempo de desenvolvimento e necessidade de testes de confiabilidade.

4. Aquisição baseada em software

O artigo discute web crawling e OPC UA. Ferramentas como Scrapy e Selenium atendem aplicações web, mas não resolvem diretamente softwares industriais locais do tipo client/server.

OPC UA melhora a interoperabilidade, mas ainda exige suporte do equipamento e conhecimento dos drivers, parâmetros, formatos e protocolos. Tecnologias como MQTT e AMQP também precisam ser

compreendidas e configuradas.

A conclusão dos autores é que essas abordagens continuam exigindo forte customização para diferentes dispositivos.

5. Principal contribuição

A principal contribuição é um framework de aquisição que tenta ignorar a heterogeneidade da camada inferior. O sistema não depende de entender o protocolo interno do equipamento sempre que a informação necessária puder ser observada visualmente.

A captura pode vir de screenshot, gravação de tela, vídeo ou câmera externa. O YOLOv8 detecta apenas a região relevante e o OCR interpreta o valor, evitando aplicar reconhecimento sobre a imagem inteira.

Fluxo central: equipamento -> imagem/tela -> YOLOv8 detecta a região -> OCR reconhece o conteúdo -> dados são salvos -> integração com banco ou plataforma corporativa.

6. Trabalhos relacionados

A revisão inclui aplicações de DAQ em soldagem, máquinas de construção, ventilação mecânica em UTI, monitoramento ambiental e sistemas sísmicos.

Um exemplo discutido é o CARENet, que coleta dados de ventiladores, transmite por Wi-Fi e os armazena em servidores com uma plataforma de gerenciamento.

O argumento do artigo é que muitos trabalhos anteriores obtêm bons resultados em suas aplicações específicas, mas não foram concebidos como soluções universais; por isso, tendem a ter custo alto e baixa generalização.

7. Por que YOLOv8

YOLO significa You Only Look Once e transforma a detecção de objetos em um processo de ponta a ponta. No artigo, o 'objeto' procurado não é uma pessoa ou veículo, mas uma região da interface que contém o dado de interesse.

Os autores destacam velocidade, precisão e flexibilidade. A detecção rápida permite identificar a região relevante em tempo real antes de enviar somente esse recorte para o OCR.

8. Papel do OCR

O OCR é responsável por transformar a imagem recortada em texto utilizável. O problema industrial é particularmente sensível porque números e unidades fazem parte da medição.

Erros como confundir 'u' com o símbolo micro podem mudar a interpretação de uma grandeza. Por isso, os autores treinam um OCR especializado em vez de depender somente de um modelo genérico.

9. Arquitetura geral proposta

A arquitetura aceita dados capturados por screenshot, vídeo ou câmera. Isso permite coletar informações até de equipamentos antigos, sem rede moderna ou API disponível.

Após o reconhecimento, os resultados podem ser gravados em arquivos, bancos de dados ou enviados para a plataforma de Big Data da empresa.

10. Processo completo de aquisição

O processo começa com a construção e rotulagem de um dataset. As regiões da tela que devem ser coletadas são marcadas manualmente.

Depois o YOLOv8 é treinado para reconhecer essas regiões. Em operação, imagens ou vídeos são enviados ao modelo, as regiões são detectadas, recortadas e encaminhadas ao OCR.

O texto reconhecido passa por verificação, armazenamento e, quando necessário, envio para a plataforma corporativa.

11. Construção do dataset do YOLO

As regiões relevantes foram rotuladas com o LabelImg. O artigo usa exemplos de equipamentos de teste de papel para embalagem, com áreas designadas como Area1 e Area2.

As imagens foram geradas em diferentes condições de janela, inclusive maximizada e não maximizada, para que o modelo aprendesse que a posição absoluta da informação pode variar.

12. Equipamentos usados nos testes

O sistema foi testado em 16 tipos de equipamentos, totalizando 30 dispositivos. São mencionadas máquinas de tração, máquinas de compressão, sistemas HPLC e medidores de espessura.

Foram coletadas cerca de 400 imagens por equipamento. Em um dos exemplos detalhados, 300 imagens foram usadas para treinamento e 100 para teste.

O pré-processamento incluiu ajuste de contraste, redução de ruído e aumento de nitidez para reduzir o efeito de baixa iluminação, borramento e outras degradações.

13. Métricas utilizadas para avaliar o YOLO

A seleção do modelo considera precisão, recall, mAP@0.5, mAP@0.5:0.95, complexidade em GFLOPs, tempo de inferência e número de parâmetros.

As métricas de detecção medem tanto a capacidade de encontrar corretamente as regiões quanto o custo computacional necessário para isso.

Precisão: $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

Recall: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

14. Comparação entre modelos YOLO

Os autores compararam diferentes variantes e escolheram o YOLOv8n como o melhor compromisso entre desempenho e custo computacional.

O YOLOv8n apresentou aproximadamente 8,195 GFLOPs e tempo de inferência de 1,491 ms. O YOLOv8s ficou em torno de 3,022 ms, aproximadamente o dobro no teste apresentado.

Mesmo com menor complexidade, o YOLOv8n manteve desempenho competitivo nas métricas de detecção e foi escolhido para operação em tempo real.

15. Hardware e tempo de treinamento

Os experimentos foram executados em uma estação com GPU NVIDIA RTX 4090 D e 64 GB de RAM.

O treinamento do YOLOv8n levou aproximadamente 5,26 horas. Teste e validação levaram aproximadamente 1,01 hora cada.

16. Resultado do YOLO

O modelo conseguiu detectar corretamente regiões como Area1 e Area2, inclusive quando havia sobreposição de marcações. Isso sustentou o uso do YOLO como etapa de localização antes do OCR.

17. Desenvolvimento do OCR

Para gerar dados de treinamento, os autores utilizaram o Text Renderer, ferramenta capaz de produzir imagens sintéticas com variações de fonte, layout, sombras, bordas, contraste e outras características.

Esse método permite criar grandes volumes de exemplos sem fotografar manualmente cada combinação.

18. Dataset do OCR

O conjunto de treinamento do OCR foi dividido em duas partes: cerca de 500 mil textos com caracteres chineses e cerca de 500 mil amostras contendo letras, números e unidades.

O treinamento utilizou batch size 64 e embaralhamento dos dados para evitar efeitos indesejados de ordenação.

19. Arquitetura do modelo OCR

O modelo se baseia principalmente no PP-OCRv2. As imagens são convertidas para escala de cinza antes de entrar na rede.

A estrutura inclui camadas convolucionais para extração de características, formação de sequências, rede LSTM bidirecional e transcrição final do texto reconhecido.

20. Comparação dos modelos OCR

Modelo	Precisão	Tempo	Tamanho
Cncor	76,90%	706,67 s	2,47 GB
EasyOCR	67,98%	2079,04 s	2,46 GB
PaddleOCR	87,30%	1229,29 s	2,87 GB
Pytesseract	36,73%	885,66 s	172 MB
Modelo treinado	95,70%	1032,76 s	2,15 GB

O modelo desenvolvido pelos autores alcançou 95,70% de precisão, cerca de 8 pontos percentuais acima do PaddleOCR, e foi aproximadamente 196,53 s mais rápido que ele no teste apresentado.

21. Problema importante encontrado pelo OCR

Modelos genéricos tendem a funcionar melhor quando o conteúdo é homogêneo, por exemplo apenas inglês ou apenas chinês. Quando há mistura de idiomas, números e unidades, a taxa de erro aumenta.

A dificuldade em distinguir caracteres semelhantes ou símbolos de unidade reforça a importância de treinar o OCR com dados próximos do contexto real de uso.

22. Idiomas diferentes exigem novo treinamento

O artigo reconhece que, devido à diversidade de idiomas, a adaptação para um novo idioma pode exigir retreinamento. Isso melhora a precisão e também pode manter o modelo mais enxuto e especializado.

23. Implantação prática na empresa

A empresa utilizava 16 tipos de equipamentos, totalizando 30 dispositivos conectados por uma rede com switch TP-LINK de 48 portas.

A solução segue arquitetura cliente/servidor: os clientes enviam imagens ou vídeos e o servidor executa detecção, OCR e armazenamento. Isso reduz a exigência computacional dos equipamentos de campo.

24. Arquitetura em cinco camadas

O sistema é organizado em camada física, camada de aquisição de dados, camada de interação de rede, camada de gerenciamento de dados e camada de aplicação.

Essa organização separa claramente dispositivos, captura, rede, armazenamento/servidor e uso final dos dados para análise e decisão.

25. Segurança dos dados

Os autores mencionam criptografia AES-256 para dados transmitidos entre equipamentos e servidor.

Também são citados controle de acesso pelo princípio do menor privilégio, detecção de intrusão, auditorias periódicas e um plano de resposta a incidentes.

26. Funcionamento em tempo real

Durante os testes, a janela com os resultados foi movimentada na tela. O YOLO continuou acompanhando a região, mostrando que o sistema não depende apenas de coordenadas fixas.

Os resultados reconhecidos pelo OCR eram exibidos e salvos localmente em arquivo Result.txt.

27. Escalabilidade

Embora o experimento tenha usado 30 dispositivos, os autores discutem cenários com centenas ou milhares de equipamentos.

Nesse caso, seriam necessários recursos como computação distribuída, otimização de algoritmos, compressão, redundância e failover para lidar com processamento, largura de banda e latência.

28. Principal conclusão do artigo

A proposta pode ser resumida como 'se o dado pode ser visto, ele pode ser coletado'. O sistema evita, em muitas situações, a necessidade de acessar firmware, capturar pacotes ou implementar drivers e protocolos proprietários.

O maior ganho é a generalização entre equipamentos diferentes, especialmente quando o acesso ao protocolo interno é inviável.

29. Limitação mais importante

O OCR não atinge 100% de precisão. Mesmo com 95,70% no teste, ainda existe risco de erro de interpretação.

Para aplicações críticas, pode ser necessário treinar modelos específicos e incluir validação adicional dos dados reconhecidos.

30. Trabalhos futuros

Os autores planejam incorporar outros tipos de dados, como áudio, melhorar os algoritmos de aquisição e expandir a arquitetura com computação em nuvem.

O objetivo é aumentar escala, precisão, eficiência e número de aplicações industriais.

Síntese final

O trabalho cria um 'leitor visual universal' para equipamentos industriais. Em vez de desenvolver uma integração específica para cada máquina, captura-se a tela ou imagem, usa-se YOLOv8 para localizar o campo relevante, OCR para ler o valor e uma arquitetura cliente/servidor para armazenar e integrar os dados. O YOLOv8n foi escolhido pela combinação de velocidade e baixo custo computacional, enquanto o OCR treinado atingiu 95,70% de precisão. O sistema foi testado em 30 dispositivos de 16 tipos diferentes, demonstrando o potencial de generalização.